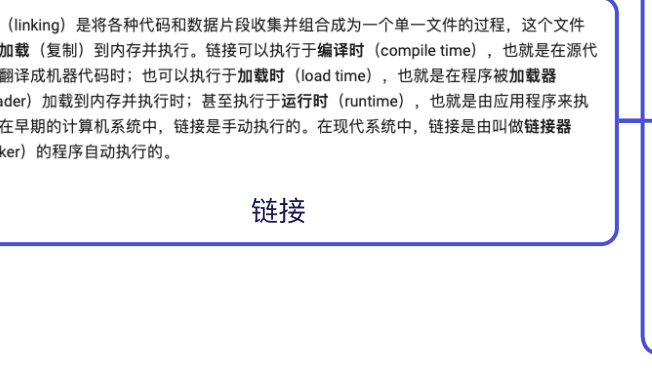
内存管理 @Beokayy_

装入和链接 所有教材都写的超级无敌模糊，看看得了

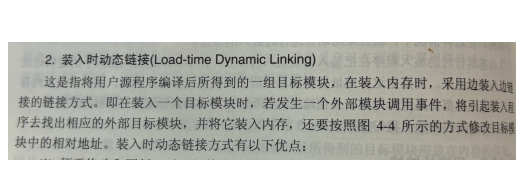
- 装入
  - 绝对装入
    - 事先知道程序将驻留在内存的什么位置
    - 程序中的虚拟地址和实际物理地址完全相同
    - 只适用于单道程序环境
  - 可重定位装入（静态地址重定位）
    - 根据内存当前的情况，将装入模块装入到内存的适当位置
    - 装入时对地址进行重定位，即完成虚拟地址到物理地址的映射
    - 地址变换在装入时一次性完成，以后不再改变，因此程序运行时不能在内存中移动位置，也就是说静态重定位就是给没有对换功能的存储系统准备的
    - 没有对换功能的内存管理方式采用静态重定位
  - 动态运行时装入（动态地址重定位）
    - 在程序执行过程中进行地址变换，每次访问存储单元时才将虚拟地址转换为物理地址
    - 为了使地址转换不影响指令的执行速度，需要重定位寄存器的支持
    - 程序在内存中不必是连续的，一个程序可分为若干个目标模块，分别装入不相邻的存储区域
    - 请求分页、动态分区分配（紧凑技术）采用动态重定位



链接（linking）是将各种代码和数据片段收集并组合成为一个单一文件的过程，这个文件可被**加载**（复制）到内存并执行。链接可以执行于**编译时**（compile time），也就是在源代码被翻译成机器代码时；也可以执行于**加载时**（load time），也就是在程序被**加载器**（loader）加载到内存并执行时；甚至执行于**运行时**（runtime），也就是由应用程序来执行。在早期的计算机系统中，链接是手动执行的。在现代系统中，链接是由叫做**链接器**（linker）的程序自动执行的。

链接

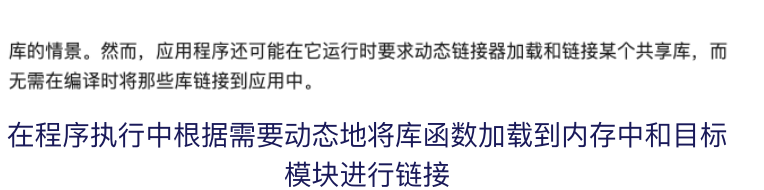
- 链接
  - 静态链接
    - 在程序运行之前，先将各目标模块及它们所需的库函数，链接成一个完整的装入模块，以后不再拆开
  - 装入时动态链接
    - 将用户源程序编译后所得到的一组目标模块，在装入内存时，采用边装入边链接的链接方式
      - 到底是全部调入还是根据调用情况调入？压根不是重点啊，别纠结了😠



2. 装入时动态链接(Load-time Dynamic Linking)

这是指将用户源程序编译后所得到的一组目标模块，在装入内存时，采用边装入边链接的链接方式。即在装入一个目标模块时，若发生一个外部模块调用事件，将引起装入程序去找出相应的外部目标模块，并将它装入内存，还要按照图 4-4 所示的方式修改目标模块中的相对地址。装入时动态链接方式有以下优点：

  - 运行时动态链接



库的情景。然而，应用程序还可能在它运行时要求动态链接器加载和链接某个共享库，而无需在编译时将那些库链接到应用中。

    - 在程序执行中根据需要动态地将库函数加载到内存中和目标模块进行链接

内存分配

- 连续分配
  - 单一连续分配
    - 单道程序环境
    - 内存分为系统区和用户区
    - 用户区被一道用户程序独占
  - 固定分区分配
    - 把整个用户空间分成几个固定大小区域，每个分区只装一个作业

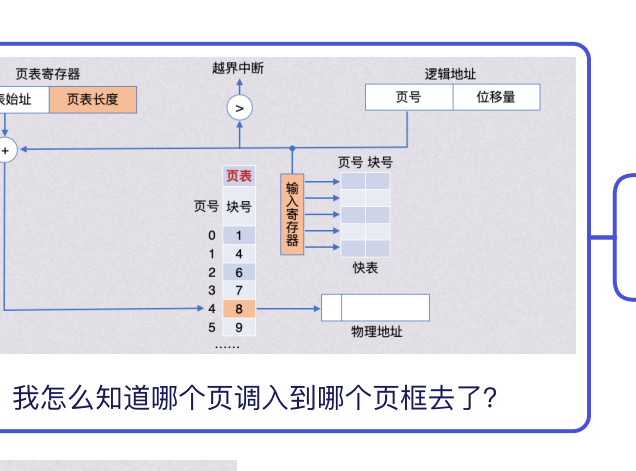
分区号	大小/KB	起始地址/KB	状态
1	1	20	已分配
2	4	21	已分配
3	32	25	已分配

    - 分区使用表，按分区大小排队，各表项包括对应分区的起始地址、大小、状态

分区号	分区大小(KB)	分区始址	状态
1	50	85	空闲
2	32	155	空闲
3	70	275	空闲
4	60	532	空闲
5	...	...	...

空闲分区表

  - 动态分区分配
    - 分配算法
      - 首次适应：空闲分区按照地址递增次序排列，每次分配内存，顺序查找第一个能满足大小的空闲分区，分配给作业
      - 邻近适应(循环首次适应算法)：相比于首次适应，改为分配内存时从上次查找结束的位置开始继续查找
      - 最佳适应：空闲分区按照容量递增排列，每次分配内存的时候找到最小的且能满足大小的空闲分区
      - 最坏适应：空闲分区按照容量递减的次序排列，每次分配内存的时候找最大的空闲分区分配
      - 伙伴系统：伙伴系统规定：所有分区均为2的k次幂，k是正整数。所有相同大小的空闲分区，为他们单独设立一个空闲分区双向链表。需要给进程分配大小为n的空间时，先计算$2^{i-1} < n \le 2^i$,在分区大小为2^i的空闲分区链表中找，找到就分配。找不到就在大小2^{i+1}的空闲分区链表中找，如果找到，把该分区分成两个相同大小分区，称为一对伙伴。其中一个分区用于分配，再把另一个分区加入大小为2^i的空闲分区链表中
    - 紧凑技术
      - 为了整合外部碎片，动态分区分配必须配合紧凑技术
      - 需要动态重定位寄存器的支持
    - 分区的分配和回收
      - 分配：根据系统的分配算法，找到空闲分区表中对应的分区，如果进程需要空间<空闲分区空间，则把空闲分区分割后，多余的空闲空间留在空闲分区表中，根据需求排序，其余的分配给发出申请的进程
      - 回收：进程运行完毕释放内存时，系统根据回收区的首址，从空闲表中找到对应插入点，根据情况选择是否与其他空闲分区合并
- 离散分配
  - 分页
    - 将主存和进程划分为大小相等的页框和页，将一个进程的所有页离散地调入页框



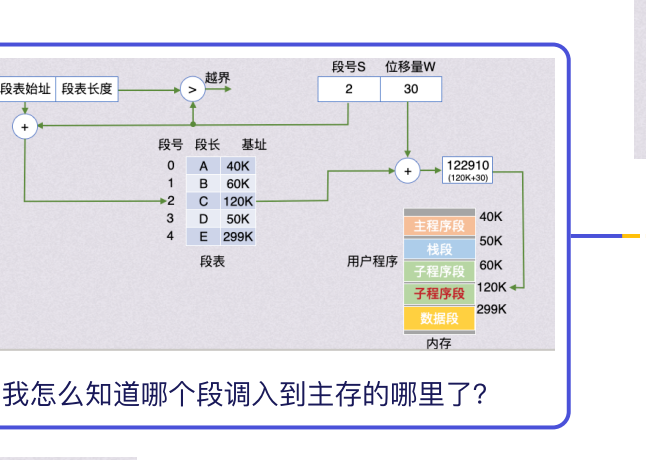
我怎么知道哪个页调入到哪个页框去了?

页号	块号
0	1
1	4
2	6
3	7
4	8
5	9
……	

    - 页表
      - 一个进程对应一张页表
      - 页表起始地址放在PCB中
    - TLB（快表）

页号	页内偏移量

    - 地址结构
  - 分段
    - 从程序员的角度提出，为了方便编程，可将进程按照逻辑分段（段的大小不一致），不对主存做任何划分！！！



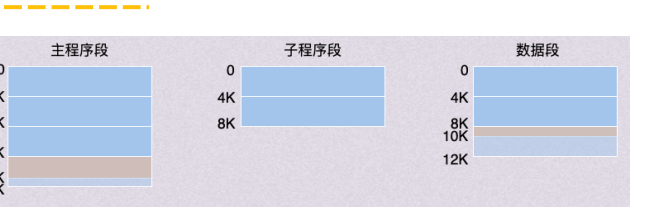
我怎么知道哪个段调入到主存的哪里了?

段号	段长	基址
0	A	40K
1	B	60K
2	C	120K
3	D	50K
4	E	299K

段表

段号	段内地址

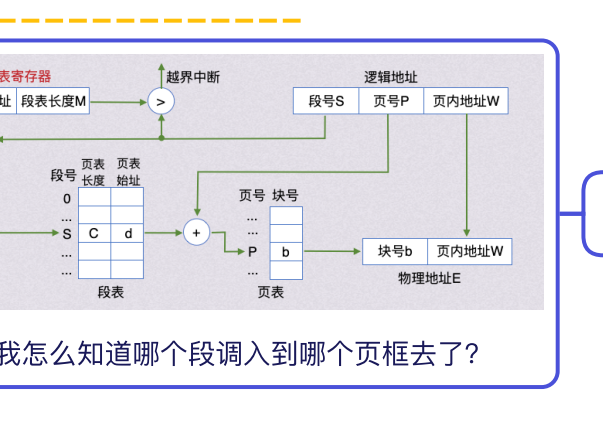
    - 地址结构
  - 段页式



    - 先把用户程序分成段，再把每个段分成页，并为每个段赋予一个段名，主存则划分页框

段号S	段内页号P	页内地址W

    - 地址结构



我怎么知道哪个段调入到哪个页框去了?

    - 每个进程建立一张段表，每个段对应一个段表项，段表项至少包括段号、页表长度、页表始址
    - 每个段有一张页表，每个页表项至少包括页号和页框号

页、页框、页表、页表项

- 页与页框
  - 页=页框
- 页与页表项
  - 页表项保存的是页号到页框号的映射
  - 一个进程有多少页，页表里就有多少个页表项
- 页表与页表项
  - 页表由页表项组成
- 页表与页框
  - 页表放在主存内核区，因为主存被划分为页框，因此页表就在页框里
  - 普通页表可以占多个页框
多级页表的每个页表最多只能占一个页框

多级页表

- 构造
  - 将页表（二级页表）分页，离散的存放在内存里，再来一张表（一级页表、顶级页表、页目录）记录它们的位置
- 特点
  - 多级页表的每个页表最多都只能占一个页框
  - 顶级页表的页表项个数=二级页表的个数

页目录号	页号	页内偏移

- 地址结构
  - 根据页目录号字段确定顶级页表中的表项
这个表项存放了二级页表所在页框号
  - 根据页号字段确定二级页表中的表项
这个表项存放了要找的页所在的页框号
  - 将页框号拼上页内偏移字段即可得到物理地址
  - 页目录号有n位，则页目录有2^n个表项
  - 页号有n位，则每张二级页表有2^n个表项

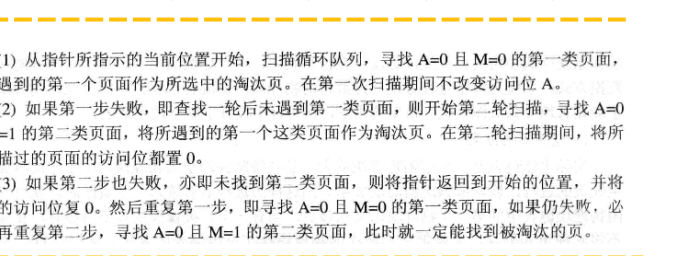
页目录号+页号就是划分多级页表前的虚页号

碎片问题

- 内部碎片
  - 定义
    - 给进程分的空间>进程需要的空间
  - 常见于
    - 基本分页
    - 单一连续分配
    - 固定分区分配
  - 不会产生内部碎片
    - 基本分段
    - 动态分区分配（有可能产生，但可能性较小）
- 外部碎片
  - 内存中因为太小没办法分配出去的空间
  - 常见于
    - 动态分区分配
  - 不会产生内部碎片
    - 基本分页
    - 固定分区分配

虚拟存储 （除去计组已讲过的部分）

- 内存分配
  - 固定分配
    - 为每个进程分配固定数目的物理块，进程运行期间不再改变
  - 可变分配
    - 先为每个进程分配一些物理块，运行期间可以根据情况增加减少
  - 局部置换
    - 如果进程在运行中发现了缺页，只能从分配给该进程的n个页框中选择一页换出再调入一页，这样来保证分配给该进程的内存空间不变
  - 全局置换
    - 如果进程运行过程中发现缺页，OS会在空闲页框中取出一块分配给该进程，这种策略下凡产生缺页的进程都会获得新的页框，空闲页框队列中的页框用完时OS才能从内存中选择一页调出，被选择调出的页可能是任何一个进程的，被选中的进程拥有的页框会减少
  - 策略
    - 固定分配局部置换
    - 可变分配全局置换
    - 可变分配局部置换
- 调页
  - 对换区
    - 对换区通常用连续分配方式组织磁盘块，因此读写速度更快
    - 若对换区空间足够，所需页面将从对换区调入，在进程运行前需将与该进程有关的文件从文件区拷贝到对换区
  - 文件区
    - 文件区通常用离散分配方式组织磁盘块，因此读写速度更慢
    - 若系统缺少足够的对换区空间，凡是不会被修改的文件都直接从文件区调入，这样换出的时候不用写回磁盘，而对于可能被修改的部分换出的时候调到对换区，以后需要再从对换区调入
  - UNIX方式
    - 凡是未运行过的页面都从文件区调入
    - 运行过被换出的页面放在对换区，下次从对换区调入
    - 允许页面共享，进程请求的页面可能已被其他进程调入内存，此时就无需从对换区调入
- 页面置换算法
  - 最佳OPT置换算法
    - 活在理论里的算法，每次选择未来最长时间不会被访问的页面进行淘汰
    - 因为无法预知未来，所以暂时无法实现
  - 先进先出FIFO置换算法
    - 每次淘汰在内存里驻留最久的页面
    - FIFO算法可能发生Belady异常（为进程分配的物理块增多，缺页次数不减反增的异常现象）
  - 最近最久未使用LRU置换算法
    - 因为无法预测未来页面的使用情况，所以只能用过去的情况作未来情况的近似，LRU算法就是每次选择最近最久未被使用的页面淘汰
  - 时钟CLOCK置换算法
    - 给所有页面设置访问位，将内存中的所有页链接成一个循环队列，绕圈检查
    - 访问位=0则换出
访问位=1则置0后扫描下一页
    - 某一页被替换时，指针会指向被替换页面的下一页（所以不是每次淘汰都重新从队头开始扫描）
  - 改进型CLOCK置换算法
    - （访问位，修改位）
    - 淘汰优先级：（0，0）>（0，1）>（1，0）>（1，1）

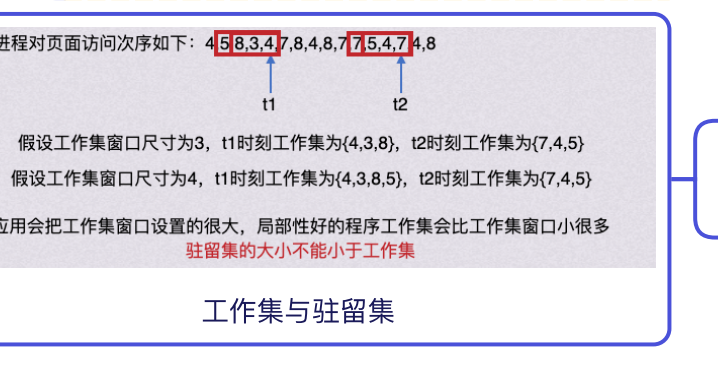


(1) 从指针所指示的当前位置开始，扫描循环队列，寻找 A=0 且 M=0 的第一类页面，将所遇到的第一个页面作为所选中的淘汰页。在第一次扫描期间不改变访问位 A。

(2) 如果第一步失败，即查找一轮后未遇到第一类页面，则开始第二轮扫描，寻找 A=0 且 M=1 的第二类页面，将所遇到的第一个这类页面作为淘汰页。在第二轮扫描期间，将所有扫描过的页面的访问位都置 0。

(3) 如果第二步也失败，亦即未找到第二类页面，则将指针返回到开始的位置，并将所有的访问位复 0。然后重复第一步，即寻找 A=0 且 M=0 的第一类页面，如果仍失败，必要时再重复第二步，寻找 A=0 且 M=1 的第二类页面，此时就一定能找到被淘汰的页。

- 抖动
  - 页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动
  - 根本原因是分配给每个进程的物理块太少，已经不能满足进程正常运行的基本要求，每个进程运行时频繁的页面调度导致CPU利用率急剧下降



某进程对页面访问次序如下：4,5,8,3,4,7,8,4,8,7,7,5,4,7,4,8

假设工作集窗口尺寸为3，t1时刻工作集为(4,3,8)，t2时刻工作集为(7,4,5)

假设工作集窗口尺寸为4，t1时刻工作集为(4,3,8,5)，t2时刻工作集为(7,4,5)

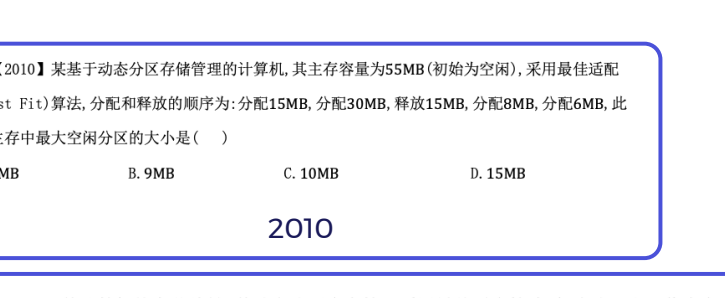
应用会把工作集窗口设置的很大，局部性好的程序工作集会比工作集窗口小很多

驻留集的大小不能小于工作集

工作集与驻留集

- 工作集
  - 某段时间间隔内，进程要访问的页面集合
- 驻留集
  - 给进程分配物理块的多少

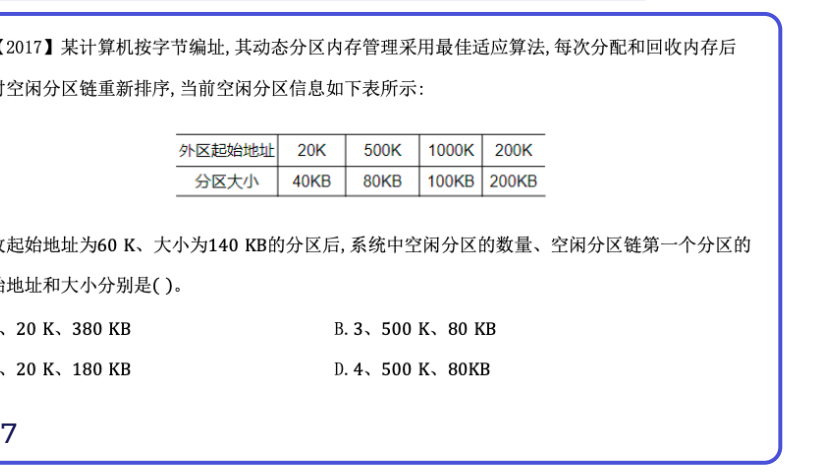
真题



2.【2010】某基于动态分区存储管理的计算机，其主存容量为55MB(初始为空闲)，采用最佳适配(Best Fit)算法，分配和释放的顺序为：分配15MB，分配30MB，释放15MB，分配8MB，分配6MB，此时主存中最大空闲分区的大小是(　)

A. 7MB　B. 9MB　C. 10MB　D. 15MB

2010



4.【2017】某计算机按字节编址，其动态分区内存管理采用最佳适应算法，每次分配和回收内存后都对空闲分区链重新排序，当前空闲分区信息如下表所示：

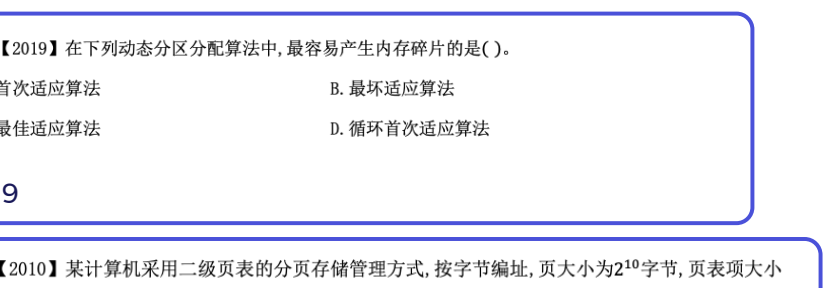
分区起始地址	20K	500K	1000K	200K
分区大小	40KB	80KB	100KB	200KB

回收起始地址为60 K、大小为140 KB的分区后，系统中空闲分区的数量、空闲分区链第一个分区的起始地址和大小分别是( )。

A. 3、20 K、380 KB　B. 3、500 K、80 KB

C. 4、20 K、180 KB　D. 4、500 K、80KB

2017

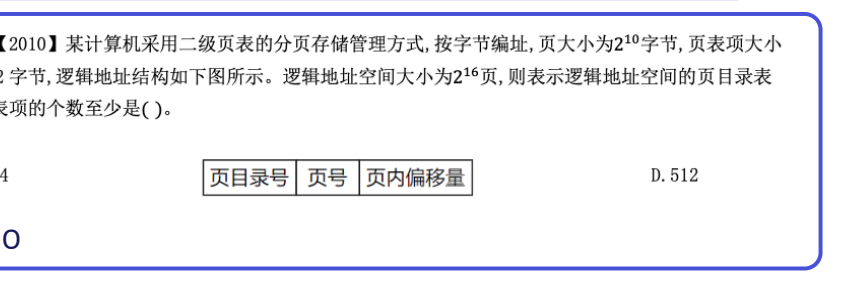


5.【2019】在下列动态分区分配算法中，最容易产生内存碎片的是( )。

A. 首次适应算法　B. 最坏适应算法

C. 最佳适应算法　D. 循环首次适应算法

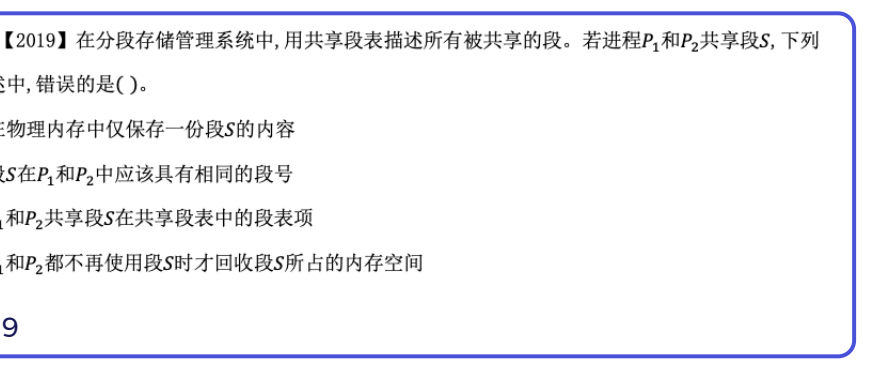
2019



7.【2010】某计算机采用二级页表的分页存储管理方式，按字节编址，页大小为2^{10}字节，页表项大小为2字节，逻辑地址结构如下图所示。逻辑地址空间大小为2^{16}页，则表示逻辑地址空间的页目录表中表项的个数至少是( )。

A. 64　| 页目录号 | 页号 | 页内偏移量 |　D. 512

2010



10.【2019】在分段存储管理系统中，用共享段表描述所有被共享的段。若进程P_1和P_2共享段S，下列叙述中，错误的是( )。

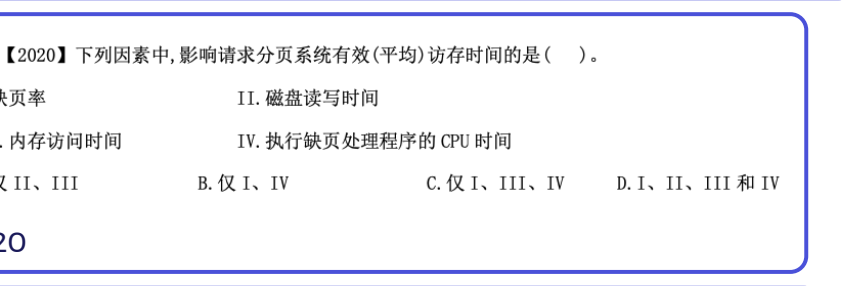
A. 在物理内存中仅保存一份段S的内容

B. 段S在P_1和P_2中应该具有相同的段号

C. P_1和P_2共享段S在共享段表中的段表项

D. P_1和P_2都不再使用段S时才回收段S所占的内存空间

2019



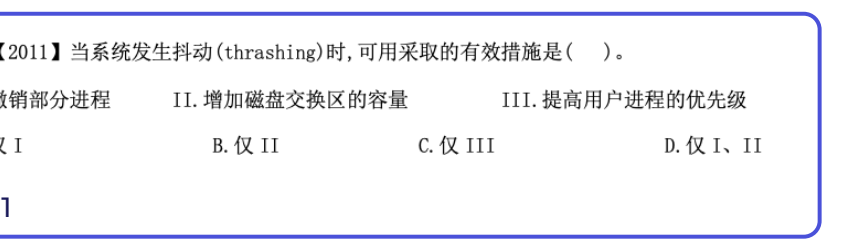
12.【2020】下列因素中，影响请求分页系统有效(平均)访存时间的是(　)。

I. 缺页率　II. 磁盘读写时间

III. 内存访问时间　IV. 执行缺页处理程序的 CPU 时间

A. 仅 II、III　B. 仅 I、IV　C. 仅 I、III、IV　D. I、II、III 和 IV

2020

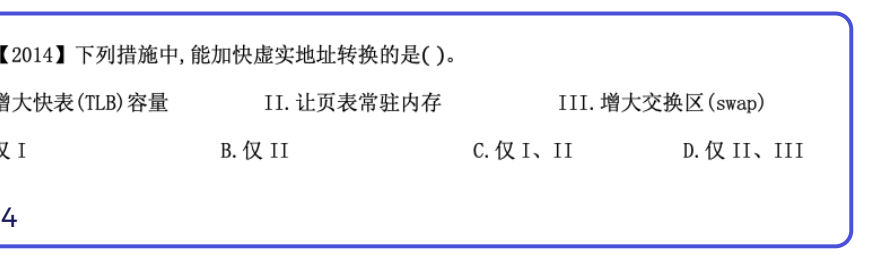


2.【2011】当系统发生抖动(thrashing)时，可用采取的有效措施是(　)。

I. 撤销部分进程　II. 增加磁盘交换区的容量　III. 提高用户进程的优先级

A. 仅 I　B. 仅 II　C. 仅 III　D. 仅 I、II

2011

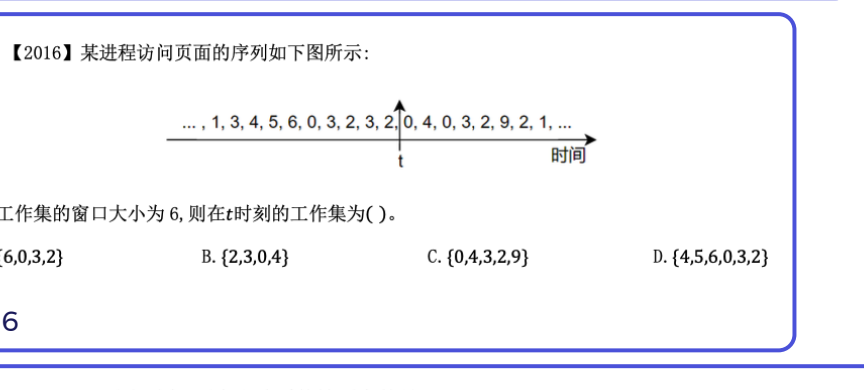


5.【2014】下列措施中，能加快虚实地址转换的是( )。

I. 增大快表(TLB)容量　II. 让页表常驻内存　III. 增大交换区(swap)

A. 仅 I　B. 仅 II　C. 仅 I、II　D. 仅 II、III

2014



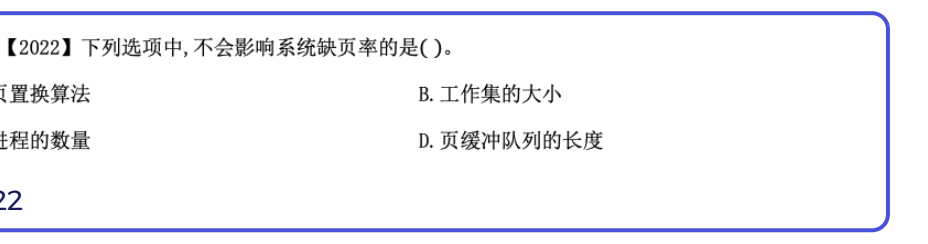
10.【2016】某进程访问页面的序列如下图所示：

…, 1, 3, 4, 5, 6, 0, 3, 2, 3, 2, 0, 4, 0, 3, 2, 9, 2, 1, …
t
时间

若工作集的窗口大小为6，则在t时刻的工作集为( )。

A. {6,0,3,2}　B. {2,3,0,4}　C. {0,4,3,2,9}　D. {4,5,6,0,3,2}

2016

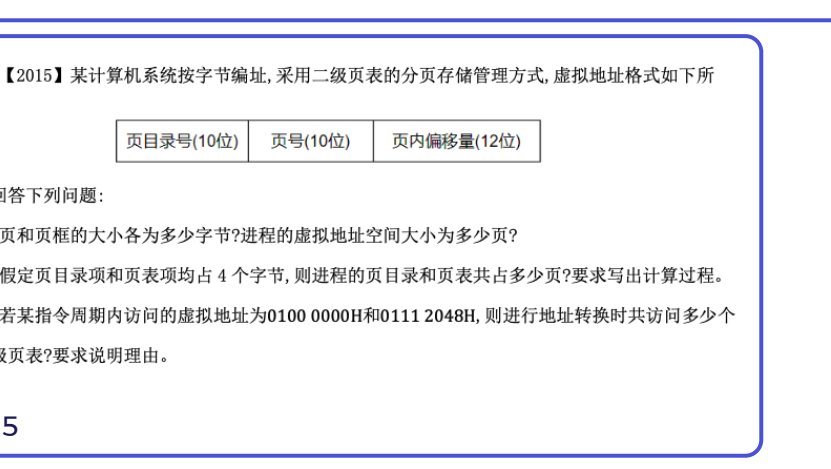


14.【2022】下列选项中，不会影响系统缺页率的是( )。

A. 页置换算法　B. 工作集的大小

C. 进程的数量　D. 页缓冲队列的长度

2022



19.【2015】某计算机系统按字节编址，采用二级页表的分页存储管理方式，虚拟地址格式如下所示：

页目录号(10位)	页号(10位)	页内偏移量(12位)

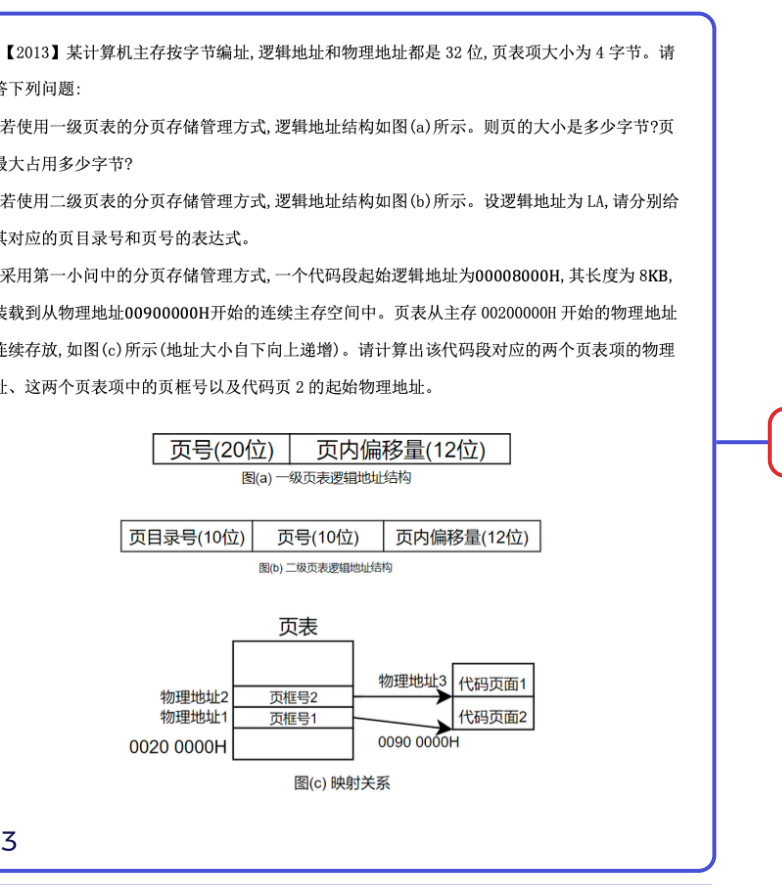
请回答下列问题：

(1) 页和页框的大小各为多少字节?进程的虚拟地址空间大小为多少页?

(2) 假定页目录项和页表项均占 4 个字节，则进程的页目录和页表共占多少页?要求写出计算过程。

(3) 若某指令周期内访问的虚拟地址为0100 0000H和0111 2048H，则进行地址转换时共访问多少个二级页表?要求说明理由。

2015



15.【2013】某计算机主存按字节编址，逻辑地址和物理地址都是 32 位，页表项大小为 4 字节。请回答下列问题：

(1) 若使用一级页表的分页存储管理方式，逻辑地址结构如图(a)所示。则页的大小是多少字节?页表最大占用多少字节?

(2) 若使用二级页表的分页存储管理方式，逻辑地址结构如图(b)所示。设逻辑地址为 LA，请分别给出其对应的页目录号和页号的表达式。

(3) 采用第一小问中的分页存储管理方式，一个代码段起始逻辑地址为00008000H，其长度为 8KB，被装载到从物理地址00900000H开始的连续主存空间中。页表从主存 00200000H 开始的物理地址处连续存放，如图(c)所示(地址大小自下向上递增)。请计算出该代码段对应的两个页表项的物理地址、这两个页表项中的页框号以及代码页 2 的起始物理地址。

页号(20位)	页内偏移量(12位)

图(a) 一级页表逻辑地址结构

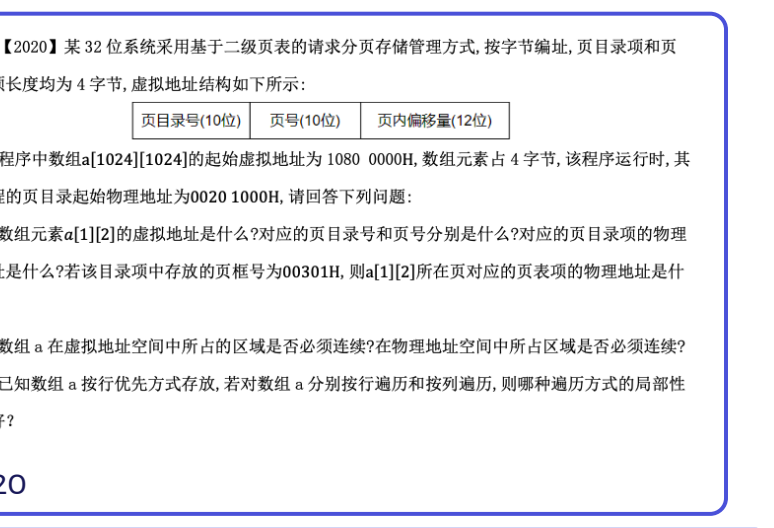
页目录号(10位)	页号(10位)	页内偏移量(12位)

图(b) 二级页表逻辑地址结构

图(c) 映射关系

2013

- 求页表项的物理地址
  - 确定页表的起始物理地址
  - 起始物理地址+页目录号*页目录项的大小
起始物理地址+页号*页表项的大小



22.【2020】某 32 位系统采用基于二级页表的请求分页存储管理方式，按字节编址，页目录项和页表项长度均为 4 字节，虚拟地址结构如下所示：

页目录号(10位)	页号(10位)	页内偏移量(12位)

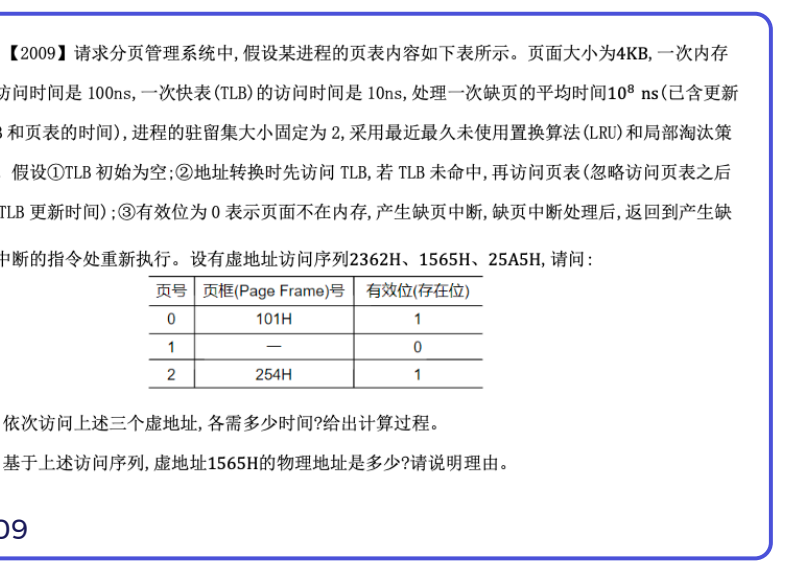
某C程序中数组a[1024][1024]的起始虚拟地址为 1080 0000H，数组元素占 4 字节，该程序运行时，其进程的页目录起始物理地址为0020 1000H，请回答下列问题：

(1) 数组元素a[1][2]的虚拟地址是什么?对应的页目录号和页号分别是什么?对应的页目录项的物理地址是什么?若该目录项中存放的页框号为00301H，则a[1][2]所在页对应的页表项的物理地址是什么?

(2) 数组 a 在虚拟地址空间中所占的区域是否必须连续?在物理地址空间中所占区域是否必须连续?

(3) 已知数组 a 按行优先方式存放，若对数组 a 分别按行遍历和按列遍历，则哪种遍历方式的局部性更好?

2020



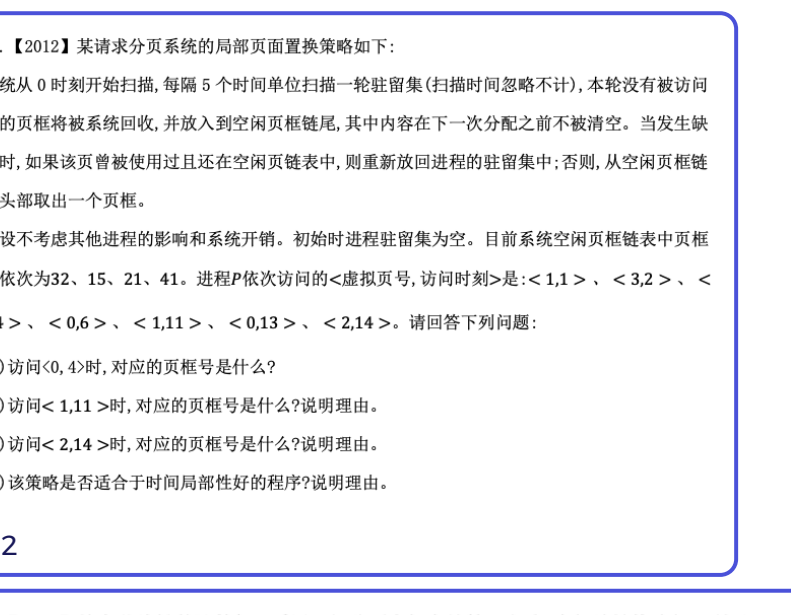
15.【2009】请求分页管理系统中，假设某进程的页表内容如下表所示。页面大小为4KB，一次内存的访问时间是 100ns，一次快表(TLB)的访问时间是 10ns，处理一次缺页的平均时间10^8 ns(已含更新 TLB 和页表的时间)，进程的驻留集大小固定为 2，采用最近最久未使用置换算法(LRU)和局部淘汰策略。假设①TLB 初始为空；②地址转换时先访问 TLB，若 TLB 未命中，再访问页表(忽略访问页表之后的 TLB 更新时间)；③有效位为 0 表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列2362H、1565H、25A5H，请问：

页号	页框(Page Frame)号	有效位(存在位)
0	101H	1
1	—	0
2	254H	1

(1) 依次访问上述三个虚地址，各需多少时间?给出计算过程。

(2) 基于上述访问序列，虚地址1565H的物理地址是多少?请说明理由。

2009



18.【2012】某请求分页系统的局部页面置换策略如下：

系统从 0 时刻开始扫描，每隔 5 个时间单位扫描一轮驻留集(扫描时间忽略不计)，本轮没有被访问过的页框将被系统回收，并放入到空闲页框链尾，其中内容在下一次分配之前不被清空。当发生缺页时，如果该页曾被使用过且还在空闲页链表中，则重新放回进程的驻留集中；否则，从空闲页框链表头部取出一个页框。

假设不考虑其他进程的影响和系统开销。初始时进程驻留集为空。目前系统空闲页框链表中页框号依次为32、15、21、41。进程P依次访问的<虚拟页号，访问时刻>是：< 1,1 >、< 3,2 >、< 0,4 >、< 0,6 >、< 1,11 >、< 0,13 >、< 2,14 >。请回答下列问题：

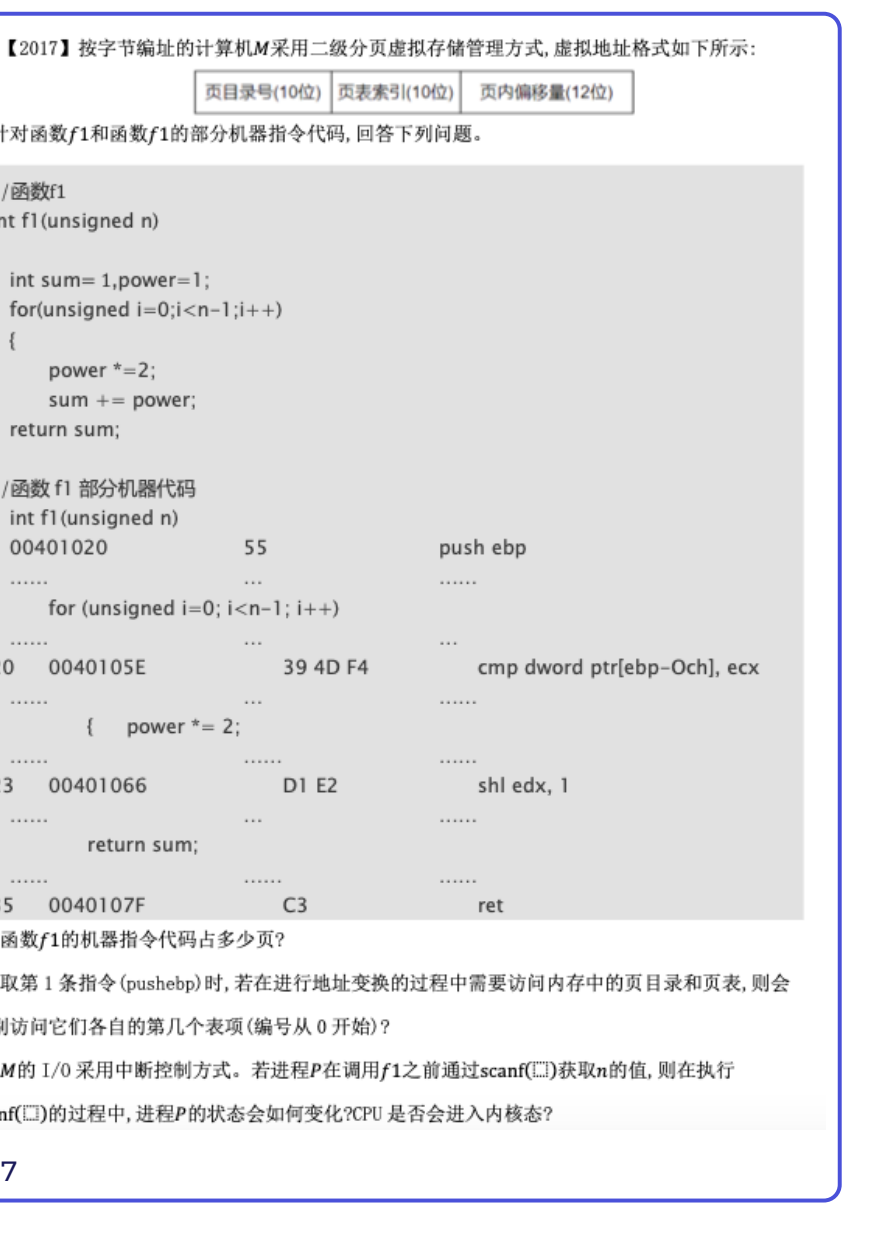
(1) 访问<0, 4>时，对应的页框号是什么？

(2) 访问< 1,11 >时，对应的页框号是什么?说明理由。

(3) 访问< 2,14 >时，对应的页框号是什么?说明理由。

(4) 该策略是否适合于时间局部性好的程序?说明理由。

2012



21.【2017】按字节编址的计算机M采用二级分页虚拟存储管理方式，虚拟地址格式如下所示：

页目录号(10位)	页表索引(10位)	页内偏移量(12位)

请针对函数f1和函数f1的部分机器指令代码，回答下列问题。

```
//函数f1
int f1(unsigned n)
{
  int sum= 1,power=1;
  for(unsigned i=0;i<n-1;i++)
  {
      power *=2;
      sum += power;
  return sum;
}
//函数 f1 部分机器指令代码
  int f1(unsigned n)
1 00401020              55                  push ebp
  ……                    …                   ……
      for (unsigned i=0; i<n-1; i++)
  ……                    …                   …
20    0040105E              39 4D F4            cmp dword ptr[ebp-0ch], ecx
  ……                    …                   ……
          {     power *= 2;
  ……                    ……                  ……
23    00401066              D1 E2               shl edx, 1
  ……                    …                   ……
          return sum;
  ……                    ……                  ……
35    0040107F              C3                  ret
```

(1) 函数f1的机器指令代码占多少页?

(2) 取第 1 条指令(push ebp)时，若在进行地址变换的过程中需要访问内存中的页目录和页表，则会分别访问它们各自的第几个表项(编号从 0 开始)?

(3) M的 I/O 采用中断控制方式。若进程P在调用f1之前通过scanf(□)获取n的值，则在执行scanf(□)的过程中，进程P的状态会如何变化?CPU 是否会进入内核态?

2017

Presented with xmind